

Assignment No 07

Text Analytics

1. Extract Sample document and apply following document preprocessing methods: Tokenization, POS Tagging, stop words removal, Stemming and Lemmatization.
2. Create representation of document by calculating Term Frequency and Inverse Document Frequency.

In [41]:

```
import nltk
import string
import math
```

In [42]:

```
nltk.download("punkt")
nltk.download("stopwords")
nltk.download("averaged_perceptron_tagger")
nltk.download("wordnet")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]   date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

Out[42]:

True

In [43]:

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.probability import FreqDist
from sklearn.feature_extraction.text import TfidfVectorizer
```

1. Extract and clean the document

In [44]:

```
with open("/content/data.txt", "r") as file:
    text = file.read()

# All ASCII Chars
alphabet = string.printable

# clean the text
def clean_data(text):
    return ''.join([word for word in text.lower() if word in alphabet])

clean_data(text)
```

Out[44]:

"python is a high-level, interpreted programming language created by guido van rossum and

first released in 1991. it is designed with an emphasis on code readability, and its syntax allows programmers to express concepts in fewer lines of code than would be possible in languages such as c++ or java.\n\npython supports multiple programming paradigms, including procedural, object-oriented, and functional programming. in simpler terms, this means its flexible and allows you to write code in different ways, whether that's like giving the computer a to-do list (procedural), creating digital models of things or concepts (object-oriented), or treating your code like a math problem (functional)."

2. Tokenization:

Breaking a text into smaller units, typically words or phrases, to facilitate further analysis.

In [45]:

```
tokens = word_tokenize(text)
print(tokens)
```

```
['Python', 'is', 'a', 'high-level', ',', 'interpreted', 'programming', 'language', 'created', 'by', 'Guido', 'van', 'Rossum', 'and', 'first', 'released', 'in', '1991', '.', 'It', 'is', 'designed', 'with', 'an', 'emphasis', 'on', 'code', 'readability', ',', 'and', 'its', 'syntax', 'allows', 'programmers', 'to', 'express', 'concepts', 'in', 'fewer', 'lines', 'of', 'code', 'than', 'would', 'be', 'possible', 'in', 'languages', 'such', 'as', 'C++', 'or', 'Java', '.', 'Python', 'supports', 'multiple', 'programming', 'paradigms', ',', 'including', 'procedural', ',', 'object-oriented', ',', 'and', 'functional', 'programming', '.', 'In', 'simpler', 'terms', 'this', 'means', 'it', 'is', 'flexible', 'and', 'allows', 'you', 'to', 'write', 'code', 'in', 'different', 'ways', ',', 'whether', 'that', 'is', 'like', 'giving', 'the', 'computer', 'a', 'to-do', 'list', '(', 'procedural', ')', ',', 'creating', 'digital', 'models', 'of', 'things', 'or', 'concepts', '(', 'object-oriented', ')', ',', 'or', 'treating', 'your', 'code', 'like', 'a', 'math', 'problem', '(', 'functional', ')', '.']
```

3. POS Parts of Speech Tagging:

Assigning grammatical categories (like noun, verb, adjective) to each word in a text based on its context and meaning.

In [46]:

```
pos_tags = nltk.pos_tag(tokens)
print(pos_tags)
```

```
[('Python', 'NNP'), ('is', 'VBZ'), ('a', 'DT'), ('high-level', 'JJ'), (',', ','), ('interpreted', 'JJ'), ('programming', 'NN'), ('language', 'NN'), ('created', 'VBN'), ('by', 'IN'), ('Guido', 'NNP'), ('van', 'NN'), ('Rossum', 'NNP'), ('and', 'CC'), ('first', 'RB'), ('released', 'VBN'), ('in', 'IN'), ('1991', 'CD'), ('.', '.'), ('It', 'PRP'), ('is', 'VBZ'), ('designed', 'VBN'), ('with', 'IN'), ('an', 'DT'), ('emphasis', 'NN'), ('on', 'IN'), ('code', 'NN'), ('readability', 'NN'), (',', ','), ('and', 'CC'), ('its', 'PRP$'), ('syntax', 'NN'), ('allows', 'VBZ'), ('programmers', 'NNS'), ('to', 'TO'), ('express', 'VB'), ('concepts', 'NNS'), ('in', 'IN'), ('fewer', 'JJR'), ('lines', 'NNS'), ('of', 'IN'), ('code', 'NN'), ('than', 'IN'), ('would', 'MD'), ('be', 'VB'), ('possible', 'JJ'), ('in', 'IN'), ('languages', 'NNS'), ('such', 'JJ'), ('as', 'IN'), ('C++', 'NNP'), ('or', 'CC'), ('Java', 'NNP'), ('.', '.'), ('Python', 'NNP'), ('supports', 'VBZ'), ('multiple', 'JJ'), ('programming', 'NN'), ('paradigms', 'NN'), (',', ','), ('including', 'VBG'), ('procedural', 'JJ'), (',', ','), ('object-oriented', 'JJ'), (',', ','), ('and', 'CC'), ('functional', 'JJ'), ('programming', 'NN'), ('.', '.'), ('In', 'IN'), ('simpler', 'NN'), ('terms', 'NNS'), (',', ','), ('this', 'DT'), ('means', 'VBZ'), ('it', 'PRP'), ('is', 'NNP'), ('s', 'VBD'), ('flexible', 'JJ'), ('and', 'CC'), ('allows', 'VBZ'), ('you', 'PRP'), ('to', 'TO'), ('write', 'VB'), ('code', 'NN'), ('in', 'IN'), ('different', 'JJ'), ('ways', 'NNS'), (',', ','), ('whether', 'IN'), ('that', 'DT'), ('is', 'VBZ'), ('like', 'IN'), ('giving', 'VBG'), ('the', 'DT'), ('computer', 'NN'), ('a', 'DT'), ('to-do', 'JJ'), ('list', 'NN'), ('(', '('), ('procedural', 'JJ'), (')', ')'), (',', ','), ('creating', 'VBG'), ('digital', 'JJ'), ('models', 'NNS'), ('of', 'IN'), ('things', 'NNS'), ('or', 'CC'), ('concepts', 'NNS'), ('(', '('), ('object-oriented', 'JJ'), (')', ')'), (',', ','), ('or', 'CC'), ('treating', 'VBG'), ('your', 'PRP$'), ('code', 'NN'), ('like', 'IN'), ('a', 'DT'), ('math', 'NN'), ('problem', 'NN'), ('(', '('), ('functional', 'JJ'), (')', ')'), ('.', '.')]
```

4. Stop Words Removal:

Eliminating common words (like "the", "is", "and") from a text that may not carry significant meaning for analysis purposes.

In [40]:

```
In [40]:
```

```
stop_words = set(stopwords.words('english'))
filtered_tokens = [word for word in tokens if word.lower() not in stop_words]

print(filtered_tokens)

# Punctuation Removal
filtered_tokens = [word for word in tokens if word.lower() not in string.punctuation]
print(filtered_tokens)
```

```
['Python', 'high-level', ',', 'interpreted', 'programming', 'language', 'created', 'Guido',
', 'van', 'Rossum', 'first', 'released', '1991', '.', 'designed', 'emphasis', 'code', 're',
'adability', ',', 'syntax', 'allows', 'programmers', 'express', 'concepts', 'fewer', 'line',
's', 'code', 'would', 'possible', 'languages', 'C++', 'Java', '.', 'Python', 'supports', 'multiple',
'programming', 'paradigms', ',', 'including', 'procedural', ',', 'object-oriented', ',', 'functional',
'programming', '.', 'simpler', 'terms', ',', 'means', "'", 'flexible', 'allows', 'write', 'code',
'different', 'ways', ',', 'whether', '"s', 'like', 'giving', 'computer', 'to-do', 'list', '(', 'procedural',
')', ',', 'creating', 'digital', 'models', 'things', 'concepts', '(', 'object-oriented', ')', ',', 'treating',
'code', 'like', 'math', 'problem', '(', 'functional', ')', '.']
['Python', 'is', 'a', 'high-level', 'interpreted', 'programming', 'language', 'created',
'by', 'Guido', 'van', 'Rossum', 'and', 'first', 'released', 'in', '1991', 'It', 'is', 'designed',
'with', 'an', 'emphasis', 'on', 'code', 'readability', 'and', 'its', 'syntax', 'allows',
'programmers', 'to', 'express', 'concepts', 'in', 'fewer', 'lines', 'of', 'code', 'than',
'would', 'be', 'possible', 'in', 'languages', 'such', 'as', 'C++', 'or', 'Java', 'Python',
'supports', 'multiple', 'programming', 'paradigms', 'including', 'procedural', 'object-oriented',
'and', 'functional', 'programming', 'In', 'simpler', 'terms', 'this', 'means', 'it', 's',
'flexible', 'and', 'allows', 'you', 'to', 'write', 'code', 'in', 'different', 'ways',
'whether', 'that', '"s', 'like', 'giving', 'the', 'computer', 'a', 'to-do', 'list',
'procedural', 'creating', 'digital', 'models', 'of', 'things', 'or', 'concepts',
'object-oriented', 'or', 'treating', 'your', 'code', 'like', 'a', 'math', 'problem',
'functional']
```

5. Stemming

Reducing words to their base or root form, typically by removing suffixes, to normalize variations of words.

```
In [47]:
```

```
porter = PorterStemmer()

stemmed_words = [porter.stem(word) for word in filtered_tokens]
print(stemmed_words)
```

```
['python', 'is', 'a', 'high-level', 'interpret', 'program', 'languag', 'creat', 'by', 'gu',
'ido', 'van', 'rosum', 'and', 'first', 'releas', 'in', '1991', 'it', 'is', 'design', 'wit',
'h', 'an', 'emphasi', 'on', 'code', 'readabl', 'and', 'it', 'syntax', 'allow', 'programm',
'to', 'express', 'concept', 'in', 'fewer', 'line', 'of', 'code', 'than', 'would', 'be', 'possibl',
'in', 'languag', 'such', 'as', 'c++', 'or', 'java', 'python', 'support', 'multipl',
'program', 'paradigm', 'includ', 'procedur', 'object-ori', 'and', 'function', 'progr',
'am', 'in', 'simpler', 'term', 'thi', 'mean', 'it', 's', 'flexibl', 'and', 'allow', 'you',
'to', 'write', 'code', 'in', 'differ', 'way', 'whether', 'that', '"s', 'like', 'give',
'the', 'comput', 'a', 'to-do', 'list', 'procedur', 'creat', 'digit', 'model', 'of', 'thing',
'or', 'concept', 'object-ori', 'or', 'treat', 'your', 'code', 'like', 'a', 'math', 'problem',
'function']
```

6. Lemmetization

Similar to stemming but aims to return the base or dictionary form of a word (lemma), considering its morphological variations.

```
In [48]:
```

```
lemmatizer = WordNetLemmatizer()

lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_tokens]

print(lemmatized_words)
```

```
['Python', 'is', 'a', 'high-level', 'interpreted', 'programming', 'language', 'created',
'by', 'Guido', 'van', 'Rossum', 'and', 'first', 'released', 'in', '1991', 'It', 'is', 'designed',
'with', 'an', 'emphasis', 'on', 'code', 'readability', 'and', 'it', 'syntax', 'a
```

llows', 'programmer', 'to', 'express', 'concept', 'in', 'fewer', 'line', 'of', 'code', 'than', 'would', 'be', 'possible', 'in', 'language', 'such', 'a', 'C++', 'or', 'Java', 'Python', 'support', 'multiple', 'programming', 'paradigm', 'including', 'procedural', 'object-oriented', 'and', 'functional', 'programming', 'In', 'simpler', 'term', 'this', 'mean', 'it', ' ', 's', 'flexible', 'and', 'allows', 'you', 'to', 'write', 'code', 'in', 'different', 'way', 'whether', 'that', '"s"', 'like', 'giving', 'the', 'computer', 'a', 'to-do', 'list', 'procedural', 'creating', 'digital', 'model', 'of', 'thing', 'or', 'concept', 'object-oriented', 'or', 'treating', 'your', 'code', 'like', 'a', 'math', 'problem', 'functional']

Part 2 : Representation of Document

Term Frequency (TF): Measuring how frequently a term occurs in a document relative to the total number of terms in that document.

Inverse Document Frequency (IDF): Measuring the rarity or commonness of a term across all documents in a corpus.

TF-IDF (Term Frequency-Inverse Document Frequency): Combining TF and IDF to evaluate the importance of a term in a document relative to a corpus.

In [49]:

```
tf = FreqDist(stemmed_words)
corpus = [text]
tfidf_vectorizer = TfidfVectorizer()
tfidf_vectorizer.fit(corpus)
idf = tfidf_vectorizer.idf_

tfidf = {word: tf[word] * idf[i] for i, word in enumerate(tf.keys())}
print("Term Frequency:", tf)
print("TF-IDF:", tfidf)
```

Term Frequency: <FreqDist with 77 samples and 107 outcomes>

TF-IDF: {'python': 2.0, 'is': 2.0, 'a': 3.0, 'high-level': 1.0, 'interpret': 1.0, 'program': 3.0, 'languag': 2.0, 'creat': 2.0, 'by': 1.0, 'guido': 1.0, 'van': 1.0, 'rossum': 1.0, 'and': 4.0, 'first': 1.0, 'releas': 1.0, 'in': 5.0, '1991': 1.0, 'it': 3.0, 'design': 1.0, 'with': 1.0, 'an': 1.0, 'emphasi': 1.0, 'on': 1.0, 'code': 4.0, 'readabl': 1.0, 'syntax': 1.0, 'allow': 2.0, 'programm': 1.0, 'to': 2.0, 'express': 1.0, 'concept': 2.0, 'fewer': 1.0, 'line': 1.0, 'of': 2.0, 'than': 1.0, 'would': 1.0, 'be': 1.0, 'possibl': 1.0, 'such': 1.0, 'as': 1.0, 'c++': 1.0, 'or': 3.0, 'java': 1.0, 'support': 1.0, 'multipl': 1.0, 'paradigm': 1.0, 'includ': 1.0, 'procedur': 2.0, 'object-ori': 2.0, 'function': 2.0, 'simpler': 1.0, 'term': 1.0, 'thi': 1.0, 'mean': 1.0, ' ': 1.0, 's': 1.0, 'flexibl': 1.0, 'you': 1.0, 'write': 1.0, 'differ': 1.0, 'way': 1.0, 'whether': 1.0, 'that': 1.0, '"s"': 1.0, 'like': 2.0, 'give': 1.0, 'the': 1.0, 'comput': 1.0, 'to-do': 1.0, 'list': 1.0, 'digit': 1.0, 'model': 1.0, 'thing': 1.0, 'treat': 1.0, 'your': 1.0, 'math': 1.0, 'problem': 1.0}

In [49]: